Code Challenge: AI-Native Engineer

This take-home challenge tests your judgement: build a small product catalogue with AI tools, record your assumptions, and be ready to defend every decision in a follow-up interview.

Scenario

An entrepreneur has big plans for their idea and has decided that something custom needs to be built. The MVP, however, is straightforward: a product catalogue on web or mobile.

Customers will search and browse the catalogue and submit an enquiry for products they want. The entrepreneur has done the market research to validate the opportunity and is getting creative work done to nail the look and feel, but wants to start the build now. They have provided acceptance criteria for the basic features.

They want a proof of concept they can interact with to start firming up the vision. Product metadata is not important yet, so using the Fake Store API is acceptable.

They have been burned by throwaway prototypes before. They do not want a proof of concept that gets thrown away; they want a foundation that can grow and evolve with the vision. For example, the product data source, look and feel, and email provider are all expected to change.

How we assess

We expect you to finish quickly with AI tools, so we assess your judgement, not how many features you ship.

- **Use AI to plan and build.** Use any AI coding tools you like. We want to see how you direct them, not whether you can type the code yourself.
- **You own every line.** AI-generated code, docs and READMEs are fine, but you must review them. In the follow-up interview we will ask you to explain and defend the decisions in your submission, including ones the AI made.
- **Make assumptions.** The acceptance criteria deliberately leave gaps. Where something is unclear, make a reasonable assumption, record it, and move on. Do not email us to ask.
- **Time-box to 2–3 hours.** If you drop or simplify a requirement, say why. Tell us how long you actually spent; going over the time-box is fine if you are honest about it.

- **Focus on the critical path.** A small, well-reasoned solution beats a large, unexplained one.

Technical constraints

1. Choose one client and one backend from the table below, in any combination. State your choice and why in the README.
2. Do not call the Fake Store API directly from the client; all product data must go through your backend.
3. If you include infrastructure, it must be compatible with AWS.
4. A reviewer must be able to run your solution locally in under 10 minutes by following your README. For web, keep setup to one or two commands where possible (for example `docker compose up`). For mobile, the app must build and run on the iOS Simulator or Android Emulator against your locally running backend.
5. Do not commit secrets. Use an example env file (for example `.env.example`) for any configuration.
6. Your solution must not depend on paid services or accounts the reviewer would need to create.

Layer	Options
Client	React web app (TypeScript preferred) · iOS app in Swift or Objective-C · Android app in Kotlin or Java
Backend	Node.js (TypeScript preferred) · PHP with Laravel

Acceptance criteria

Home page

```
GIVEN I have chosen 5 products to be "top rated"
WHEN I am viewing the home page
THEN I see those 5 featured products
```

Category page

```
GIVEN my products are organised into categories
WHEN I click on a category link
THEN I see top rated products for that category
  AND I see additional products in that category
```

Search

```
GIVEN I am looking for a specific product
  AND I have entered a search term in the search bar
WHEN I click the search button
THEN I only see products that have text matching that search term
```

Product page

```
GIVEN I am browsing the products
  AND I want to see more details about a specific product
WHEN I click on a product tile
THEN I see the product details page
```

Enquiry

```
GIVEN I am viewing a product
  AND I am interested in a product
  AND I click on the Enquire button
  AND I complete the subsequent form with my contact details
WHEN I submit the form
THEN the sales inbox will receive an email
  AND the customer will see a "Thank you" message
```

What to submit

Submit within 3 days of receiving this email by replying with your repository link. This is a take-home exercise; there is no live coding at this stage.

1. **Repository:** a public or private Git repo on GitHub, GitLab or Bitbucket. For a private repo, give access to the reviewers named in the email.
2. **Pull request:** build on a feature branch and raise a PR into `main` in your own repo. Leave it open. The PR description is part of the assessment.
3. **Source code:** the full source, including tests.
4. **README:** setup and run instructions, plus anything the reviewer needs to configure.

5. **Decisions and assumptions:** a `DECISIONS.md` (or a section in the README) covering:
   - assumptions you made where the acceptance criteria were unclear
   - requirements you dropped or simplified, and why
   - how the design supports the solution growing beyond the PoC
   - what you would do next with another day
6. **AI usage notes:** an `AI_USAGE.md` covering:
   - tools and models you used
   - any agent instruction files (for example `CLAUDE.md`, `AGENTS.md` or `.cursor/rules`), committed to the repo
   - your plan or spec, if you used one
   - 3–5 key prompts or moments that shaped the solution
   - where the AI got something wrong and how you caught it
7. **Time spent:** your actual time, roughly split across planning, building, testing and reviewing.

Selection criteria

We score five areas. Shortlisted candidates are invited to a 45-minute follow-up to walk through the submission and make a small change to it live.

Area	What we look for
Solution design	A structure that can grow beyond the PoC: clear boundaries between frontend, backend and the product data source.
Decision-making and assumptions	Sensible assumptions where the criteria are unclear, clear trade-offs, and justified cuts.
Implementation quality	Readable, consistent code with sensible error handling, validation and security basics.
Testing	Tests that cover the critical path and give confidence to change the code.
AI engineering practice	Effective direction of AI tools, critical review of their output, and ownership of the result.